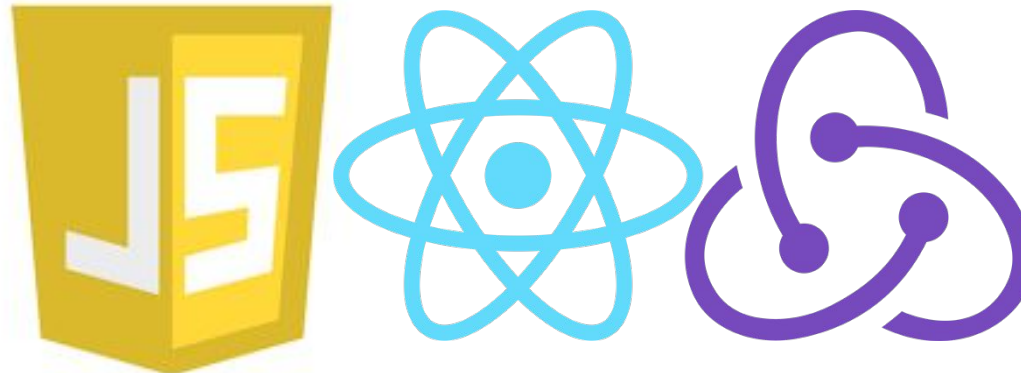


Лекція 12

Сучасний фронтенд JavaScript. React. Redux.

(Частина 1)



Транспіляція (компіляція)

Компіляція - процес перетворення коду з мови більш високого рівня в мову більш низького рівня (наприклад з JavaScript в набір машинних команд).

Транспіляція - процес перетворення коду з одної мови в іншу, котрі працюють приблизно на одному і тому ж рівні абстракції (наприклад TypeScript в JavaScript).

Таким чином **транспіляція** являється окремим випадком **компіляції**, і в подальшому буде використовуватись термін **компіляція**.

1. Введення в JavaScript.

JavaScript може виконуватись у браузері, на сервері або на будь-якому іншому пристрої який має спеціальну програму яка називається «рушієм» JavaScript.

У браузері є власний рушій, який інколи називають «віртуальна машина JavaScript».

Різні рушії мають різні «кодові імена» (V8 – в Chrome і Opera, SpiderMonkey – в Firefox)

JavaScript має власну специфікацію яка називається **ECMAScript**.

ECMAScript – стандарт, а JavaScript – самая популярная реалізація цього стандарту.

1.1. Версіонування JavaScript.

ES — це скорочення для ECMAScript.

Кожний випуск ECMAScript отримує аббревіатуру ES з його номером.

Всього існує 8 версій ECMAScript. ES1 була випущена в червні 1997 року, ES2 — в червні 1998 року, ES3 — в грудні 1999 року, і т.д.

Версія ES6/ES2015 вийшла в червні 2015 року. Це привнесло певну плутанину у зв'язку з назвою пакету, адже **ES6 і ES2015 — це одне і теж**. З виходом цього пакету оновлень комітет прийняв рішення перейти до щорічних оновлень. Тому випуск було перейменовано в ES2015, щоб відображати рік релізу. Наступні версії також називаються відповідно до року їх випуску.

ES6 = ES2015

1.2. Як виконується код?

- 1) Рушій (вбудований, якщо це браузер) зчитує («парсить») текст скрипта.
- 2) Рушій перетворює («компілює») скрипт в машинний код.
- 3) Рушій запускає машинний код.

Рушій застосовує оптимізації на кожному етапі. Він навіть переглядає скомпільований скрипт під час його роботи, аналізуючи данні які проходять через нього, і застосовує оптимізації до машинного коду.

1.3. Компілятор Babel.

Проблема: деякі браузери підтримують **НЕ** всі версії JavaScript.

Рішення: Babel компілює код, написаний на більш новій версії JS, в код більш старої версії JS

Наприклад:

Версія ES2015 (ES6):

```
const myFunc = (myParam) => console.log(myParam);
```



Версія ES5:

```
var myFunc = function myFunc(myParam) {  
    return console.log(myParam);  
};
```

2. Пам'ятка по важливим темам JS.

В інтернеті є хороший сайт, де дуже доступно і структуровано поданий матеріал по всім важливим темам, зв'язаних з мовою JavaScript:

<https://uk.javascript.info/>

В даній лекції будуть коротко викладені основні моменти, котрі потрібно завжди пам'ятати в процесі практичного написання коду.

2.1. Об'єкт {} в JS.

```
const obj = {}; // створюємо пустий об'єкт
console.log(obj.name); // undefined
console.log(obj.name.whatElse); // Uncaught TypeError:
Cannot read property 'whatElse' of undefined
```

```
obj.name = "Пацан";
console.log(obj.name); // "Пацан"
console.log(obj.name.whatElse); // undefined
```

```
const obj = { // створюємо об'єкт з полями
  age: 25,
  name: 'Пацан',
};
console.log(obj.name); // Пацан
console.log(obj.name.whatElse); // undefined
```

2.2. Об'єкт {} в JS.

```
const obj = {  
  'більше одного слова в імені поля': 'Погано, але можливо',  
};  
  
console.log(obj['більше одного слова в імені поля']); // "Погано, але  
МОЖЛИВО"
```

```
const constForField = 'field';  
const obj = {  
  [constForField]: 'значення поля',  
};  
  
console.log(obj.field); // "значення поля"  
console.log(obj[constForField]) // "значення поля"
```

2.3. ОБЪЕКТ {} В JS.

```
const privateHouse = {  
  address: 'JS street',  
  smallRoom: {  
    chairsCount: 3,  
  },  
  bigRoom: {  
    tableSize: 42,  
    sofaSize: 53,  
  },  
};  
  
console.log(privateHouse.address); // "JS street"  
console.log(privateHouse.bigRoom); // "{tableSize: 42, sofaSize: 53}"  
console.log(privateHouse.smallRoom.chairsCount); // 3
```

2.4. Об'єкт {} в JS. Неправильне копіювання.

```
const obj1 = {  
  name: 'Пацан'  
};
```

```
const obj2 = obj1;
```

```
obj2.name = 'Другий пацан';
```

```
console.log(obj2.name); // "Другий пацан"  
console.log(obj1.name); // "Другий пацан"
```

2.5. Об'єкт {} в JS. Правильне копіювання.

```
const obj1 = {  
  name: 'Пацан'  
};
```

```
const obj2 = {...obj1};
```

```
obj2.name = 'Другий пацан';
```

```
console.log(obj2.name); // "Другий пацан"
```

```
console.log(obj1.name); // "Пацан"
```

2.6. Об'єкт{} в JS. Копіювання.

```
const obj1 = {  
  name: 'Пацан',  
  age: 21  
};
```

```
console.log(obj1);  
// {name: "Пацан", age: 21}
```

```
const obj2 = {  
  ...obj1,  
  name: 'Другий пацан',  
  role: 'support'  
};
```

```
console.log(obj2);  
// {name: "Другий пацан", age: 21, role: "support"}
```

```
const obj3 = {  
  name: 'Третій пацан',  
  role: 'coder',  
  ...obj1,  
};
```

```
console.log(obj3);  
// {name: "Пацан", age: 21, role: "coder"}
```

```
const obj4 = {  
  ...obj1,  
  ...obj2,  
  ...obj3,  
};
```

```
console.log(obj4);  
// {name: "Пацан", age: 21, role: "coder"}
```

2.7. Об'єкт{} в JS. Деструктуризація.

```
const obj1 = {  
  name: 'Пацан',  
  age: 21,  
  longNameForVariableRole: 'coder',  
  car: {  
    color: 'red',  
    model: 'yellow'  
  }  
};
```

```
const {  
  name,  
  age,  
  longNameForVariableRole: role,  
  car  
} = obj1;
```

```
console.log(role); // "coder"
```

2.8. Оператори порівняння == і ===.

Завжди використовуй === замість ==

При порівнянні значень різних типів JavaScript приводить кожен з них до числа.

```
console.log('2' == '2') // true
```

```
console.log('01' == 1) // true
```

```
console.log(true == 1) // true
```

```
console.log(false == 0) // true
```

Оператор строгого порівняння === перевіряє рівність без приведення типів.

```
console.log('2' === 2) // false
```

```
console.log('01' === 1) // false
```

```
console.log(true === 1) // false
```

```
console.log(false === 0) // false
```

2.9. Логічні оператори || &&.

|| повертає **перше істине** значення або **останнє**, якщо всі були **хибними**.

```
const test1 = 0 || 'Пацан' || 'Ще пацан'; // "Пацан"  
const test2 = 0 || '' || undefined || null; // null
```

&& повертає **перше хибне** значення, або **останнє**, якщо всі були **істинні**.

```
const test1 = 'Пацан' && 0 && 'Ще пацан'; // 0  
const test2 = 'Пацан' && 'Другий пацан' && 'Ще пацан'; // "Ще пацан"
```

2.10. Функції. Стрілочні функції.

Function Declaration (Оголошення Функції)

```
function funcName(param1, param2) {  
    // some logic  
}
```

Function Expression (Функціональний Вираз)

```
const myFunc = function funcName(param1, param2) {  
    // some logic  
};
```

Основна відмінність між Function Declaration и Function Expression в тому, що **Function Declaration, створюються інтерпретатором до виконання коду**. Тому їх можна викликати до оголошення.

Arrow functions (Стрілочні функції)

```
const myFunc = (param1, param2) => {  
    // some logic  
};
```

```
const myFunc = (param1, param2) => param1 + param2; // returned logic
```

2.11. Arrays (масиви).

Створення масива

```
const arr = []; // пустий масив
```

```
const arr = [1, 2, 3] // передзаповнений масив
```

Неправильне копіювання масива

```
const arr1 = [1, 2, 3];
```

```
const arr2 = arr1;
```

```
arr2.push(4);
```

```
console.log(arr1) // 1, 2, 3, 4
```

Правильне копіювання масива

```
const arr1 = [1, 2, 3];
```

```
const arr2 = [...arr1]; // створюється новий екземпляр масива і в нього копіюються значення
```

```
arr2.push(4);
```

```
console.log(arr1); // 1, 2, 3
```

```
console.log(arr2); // 1, 2, 3, 4
```

2.11. Arrays. Основні функції.

Всі функції масивів з їх хорошим описом можна знайти за посиланням:

https://developer.mozilla.org/ru/docs/Web/JavaScript/Reference/Global_Objects/Array

Найбільш часто використовувані:

- `.map()` *// Повертає новий масив. На кожній ітерації повертає результат, який буде лежати по індексу поточного елемента*
`const arr = [1, 2, 3].map(item => item * 2); // [2, 4, 6]`
- `.filter()` *// Повертає новий масив без тих елементів, при ітерації по який повертається значення false*
`const arr = [1, 2, 3].filter(item => item !== 2); // [1, 3]`
- `.find()` *// Повертає перший елемент масива, при ітерації по якому повертається значення true. Або повертає undefined, якщо на кожній ітерації повернулось значення false (тобто елемент не було знайдено)*
`const elem = [1, 2, 3].find(item => item === 2); // 2`
`const elem = [1, 2, 3].find(item => item === 4); // undefined`
- `.some()` *// Повертає true, якщо хоч одна ітерація повернула результат true, інакше - false.*

2.12. Arrays. Основні функції.

- `.every()` *// Повертає true, якщо кожна ітерація повернула результат true, інакше - false*
- `.includes(param)` *// Повертає true, якщо хоч би 1 елемент масива рівний значенню param*

```
const elem = [1, 2, 3].includes(2); // true
```

- `.reduce()` *// Повертає акумулятор*

```
const initialResult = 0;
```

```
const maxElem = [1, 5, 2, 3].reduce(
```

```
  (tempResult, item) => { // tempResult - результат, повернений з попередньої ітерації, або initialResult, якщо це перша ітерація
```

```
    if (item > tempResult) {
```

```
      return item;
```

```
    }
```

```
    return tempResult;
```

```
  },
```

```
  initialResult // Початкове значення акумулятора
```

```
);
```

```
console.log(maxElem) // 5
```

2.13. Promises. Створення.

Promises використовуються для обробки результату виклику функцій, тривалість виконання яких наперед невідома. Наприклад, це можуть бути функції, які виконують запити на отримання даних, і ми заздалегідь не знаємо, коли ці дані прийдуть у відповідь.

Обробка полягає в тому, що коли функція, що здійснює запит даних відпрацює, вона може завершитися або успіхом (дані прийдуть), або невдачею (помилка отримання даних). І Promises надають зручний спосіб призначити код, який відпрацює у разі успіху, або у разі помилки.

```
const promise = new Promise((resolve, reject) => {  
  // Робимо якусь логіку... Наприклад, робимо запит на отримання даних.  
  // Отримуємо результат логіки  
  // Якщо результат успішний, то викликаємо resolve('результат');  
  // Якщо помилка, то викликаємо reject('текст помилки, або будь-що');  
});
```

2.14. Promises. then, catch, finally.

На практиці, найчастіше, ми вже маємо готовий об'єкт **promise**, і все, що нам залишається – це подбати про обробку результату, який поверне **promise**.

```
get('https://someAddres/someData/list') // ця функція поверне promise
  .then(result => console.log(result))    // спрацює, якщо всередині функції get()
                                          була викликана функція
                                          resolve(someResult), в яку ми помістили
                                          об'єкт someResult, і який ми отримаємо в
                                          секції .then під ім'ям result

  .catch(error => console.log(error))     // спрацює, якщо всередині функції get()
                                          була викликана функція reject(someError),
                                          в яку ми помістили об'єкт someError, і
                                          який ми отримаємо в секції .catch під
                                          ім'ям error

  .finally(() => console.log('Викликаємо цей код у будь-якому випадку')); //
спрацює завжди
```

2.15. Promises. Приклад.

```
get('https://cats/list')
  .then(catsList => {
    const catsIds = catsList.map(cat => cat.id);
    post({
      uri: 'https://cats/owners/byCatsIds',
      body: {
        catsIds,
      }
    }).then(owners => console.log(owners))
      .catch(() => console.log('error receive owners'));
  })
  .catch(() => console.log('error receive cats list'));
```

2.16. Promises. Приклад.

```
get('https://cats/list')
  .then(catsList => {
    const catsIds = catsList.map(cat => cat.id);
    return post({ // Якщо не написати return, то наступний then не
                  чекатиме завершення виконання цього promise'a
                  uri: 'https://cats/owners/byCatsIds',
                  body: {
                    catsIds,
                  }
                });
  })
  .then(owners => console.log(owners))
  .catch(() => console.log('some error occurred'));
```

2.17. Promises. Promise.all().

Promise.all(promises) чекає завершення всіх промісів, які були передані в його параметри як масив (promises) і тільки потім викликається функція .then() або .catch() і .finally()

```
const catsPromise = get('https://cats/list');
const ownersPromise = get('https://owners/list');
const promises = [
  catsPromise,
  ownersPromise,
];
```

Promise

```
.all(promises)
.then((results) => {
  const catsList = results[0];
  const ownersList = results[1];
})
.catch(() => console.log('error load or cats or owners'))
.finally(() => console.log('Цей текст з'явиться в будь якому випадку'));
```

2.18. Метод `fetch()`.

Метод **`fetch()`** дозволяє здійснювати HTTP - запити для отримання даних від сервера.

```
const requestBody = {  
  accountId: 42,  
  userId: 24  
};  
  
const url = 'https://cats/list';  
  
const options = {  
  method: 'POST',  
  headers: {  
    'Content-Type': 'application/json;charset=utf-8',  
    body: JSON.stringify(requestBody)  
  },  
};  
  
const promise = fetch(url, options);
```

2.18. Метод `fetch()`. Response.

Метод `fetch()` повертає об'єкт `promise`.

```
const promise = fetch(url, options);
```

Далі необхідно дочекатися, коли завершиться запит, щоб мати можливість вилучити з відповіді дані, які ми хочемо отримати:

```
promise.then(response => { /* ... логіка вилучення даних із відповіді */});
```

Деякі корисні поля та функції `response`, до яких ми можемо звернутися:

```
response.ok; // true, якщо HTTP-статус в діапазоні 200-299
response.status; // код статусу HTTP-запиту, наприклад 400 (помилка)
response.json(); // декодує відповідь у форматі JSON, якщо реальна відповідь не JSON, то впаде помилка
response.text(); // читає відповідь та повертає як звичайний текст
response.blob(); // повертає об'єкт як Blob (бінарні дані з типом), використовується для отримання файлів
```

2.18. Метод `fetch()`. Приклад.

```
const url =  
  'https://api.github.com/repos/javascript-tutorial/en.javascript.info/commits';  
const options = undefined;  
  
const promise = fetch(url, options);  
promise  
  .then(response => {  
    if (response.ok) {  
      response  
        .json()  
        .then(result => console.log(result))  
        .catch(() => console.log('response is not valid JSON'));  
    } else {  
      console.log('Error with status: ' + response.status);  
    }  
  });  
}
```

3. Встановлення оточення для роботи з React

- Встановлення Node.js
- Що таке npm (node package manager)
- Встановлення та запуск скелетного проекту на React
- Що таке WebPack
- Файл package.json

3.1. Встановлення Node.js.

Node.js — програмна платформа, заснована на рушії V8 (що трансліює JavaScript в машинний код), що перетворює JavaScript з вузькоспеціалізованої мови на мову загального призначення.

Завантажити пакет встановлення Node.js можна з офіційного сайту

<https://nodejs.org/ru/download/>

Після встановлення Node.js можна перевірити, чи все встановлено коректно прописавши в терміналі наступні команди:

```
C:\Users\user>node -v
v9.11.0

C:\Users\user>npm -v
5.6.0
```

3.2. Що таке npm (Node Package Manager).

Коротко, пакетом у Node.js називається один або кілька JavaScript-файлів, які представляють якусь бібліотеку або інструмент.

Node Package Manager (npm) — це стандартний менеджер пакетів, що автоматично встановлюється разом з Node.js. Він використовується для завантаження пакетів з хмарного сервера npm або для завантаження пакетів на ці сервери.

Для того, щоб завантажити необхідний пакет, потрібно виконати в терміналі команду **npm** з назвою потрібного пакета як аргумент команди, і пакет буде автоматично завантажений в поточну директорію. Наприклад:

```
> npm install moment // якщо потрібна остання версія
```

або

```
> npm install moment@2.24.0 // якщо потрібно вказати конкретну версію
```

3.3. Встановлення та запуск проекту на React.

<https://create-react-app.dev/docs/getting-started>

- 1) `npx create-react-app my-app`
- 2) `cd my-app`
- 3) `npm start`

Де **npx** - це інструмент запуску пакетів, доступний у версіях npm 5.2 та вище.